

SE1020 – Object Oriented Programming

Group Project Assignment – Grade Weightage: 10%

Software Requirements Specification

Online Movie Ticket Booking Platform

Field	Details
Project Title	Online Movie Ticket Booking Platform
Module Code	SE1020
Module Name	Object Oriented Programming
Grade Weightage	10%
Development Environment	IntelliJ IDEA
Technologies	Java, Spring Boot, JSP Servlets, HTML/CSS, Bootstrap or Tailwind CSS, JavaScript
Data Storage	File Read/Write (.txt files) — MySQL/SQL optional
Version Control	GitHub (individual commit history required)
Total Components	6 (one per group member)

Group Members & Component Assignments

#	Component	Assigned Student ID
01	User Account Management	_____
02	Movie & Showtime Management	_____
03	Seat Selection & Booking Management	_____
04	Payment & Transaction Management	_____
05	Promotion & Discount Management	_____

#	Component	Assigned Student ID
06	Review, Rating & Feedback Management	_____

1. Introduction

1.1 Project Overview

The Online Movie Ticket Booking Platform is a web-based application developed for the SE1020 Object Oriented Programming module. The system enables users to browse movies, select showtimes, choose seats, make payments, apply promotional discounts, and leave reviews — all delivered through a Java Spring Boot / JSP Servlet backend with a responsive HTML/CSS/JavaScript frontend. Data is persisted using file read/write operations on plain text files, with an optional MySQL connection.

1.2 Project Objectives

- Apply all five OOP principles: Encapsulation, Inheritance, Polymorphism, Abstraction, and Information Hiding across all six components.
- Implement all four CRUD operations (Create, Read, Update, Delete) for every component.
- Develop a user-friendly, responsive web UI using Bootstrap 5 or Tailwind CSS.
- Persist all data using file read/write operations on structured .txt files.
- Collaborate using GitHub with a clear and meaningful individual commit history per member.
- Produce complete documentation including class diagrams and a final report with GitHub commit history.

1.3 System Scope

The platform is divided into six independent but interrelated components — one per group member. Together they form a fully functional cinema ticketing system: user management, movie cataloguing, seat-based booking, payment processing, promotional discounts, and user reviews. The scope is intentionally broader than the sample Library Management System to reflect the complexity of a real-world ticketing platform.

1.4 Definitions & Acronyms

Term	Definition
CRUD	Create, Read, Update, Delete — the four standard data operations
OOP	Object-Oriented Programming
JSP	Java Server Pages — server-side templating for dynamic HTML
Servlet	Java class that processes HTTP requests and generates responses
SRS	Software Requirements Specification
MVC	Model-View-Controller architectural design pattern

Term	Definition
Admin	Privileged platform user with full management access
Regular User	Standard registered customer who browses and books tickets
Guest	Unauthenticated visitor — can browse but cannot book
TXT File Store	Plain text files used as the application data persistence layer

2. Overall System Description

2.1 System Architecture

The application follows a layered MVC (Model-View-Controller) architecture:

- **Model Layer:** Java POJO classes (User, Movie, Seat, Booking, Payment, Promotion, Review) encapsulating business data.
- **View Layer:** JSP pages with Bootstrap/Tailwind CSS providing the frontend user interface.
- **Controller Layer:** Java Servlets routing HTTP requests and delegating to service classes.
- **Service Layer:** Business logic classes encapsulating file I/O and domain rules.
- **File Storage Layer:** Text files (users.txt, movies.txt, showtimes.txt, bookings.txt, payments.txt, promotions.txt, reviews.txt) as the persistence layer.

2.2 Technology Stack

Layer	Technology
Language	Java (JDK 17+)
Backend Framework	Spring Boot / JSP Servlets
Frontend	HTML5, CSS3, JavaScript (ES6)
UI Framework	Bootstrap 5 or Tailwind CSS (group's choice)
Data Storage	File Read/Write (.txt) — MySQL optional
IDE	IntelliJ IDEA
Build Tool	Maven
Version Control	GitHub
Server	Apache Tomcat (embedded via Spring Boot)

2.3 OOP Design Principles

OOP Concept	Application in This Project
Encapsulation	All entity classes use private fields with public getters/setters — no direct field access from outside.
Inheritance	Specialised subclasses extend base classes (e.g., AdminUser extends User; PremiumBooking extends Booking).
Polymorphism	Method overriding across subclasses — e.g., calculatePrice(), getDisplayCard(), processPayment().

OOP Concept	Application in This Project
Abstraction	Abstract classes and interfaces define contracts without exposing implementation details.
Information Hiding	File I/O and business logic hidden inside service classes; only clean APIs exposed to Servlets.

3. Workload Distribution Summary

#	Component	CRUD Operations	Files Used	OOP Highlights
01	User Account Management	Register, Search, Update Profile, Delete Account	users.txt, admins.txt	Encapsulation, Inheritance (Admin/Regular), Polymorphism (authenticate)
02	Movie & Showtime Management	Add Movie, Search/Browse, Edit Details, Remove Movie	movies.txt, showtimes.txt	Encapsulation, Inheritance (Featured/Regular), Polymorphism (getDisplayCard)
03	Seat Selection & Booking	Create Booking, View History, Update Status, Cancel	bookings.txt, seats.txt	Encapsulation, Inheritance (Premium/Standard), Polymorphism (getPrice/seat types)
04	Payment & Transaction	Process Payment, View History, Refund, Delete Record	payments.txt, transactions.txt	Polymorphism (payment methods), Abstraction (IPaymentProcessor)
05	Promotion & Discount	Create Promo, Validate Code, Update Promo, Delete/Expire	promotions.txt, discounts.txt	Encapsulation, Inheritance (Seasonal/Fixed/Percentage), Polymorphism (calculateDiscount)
06	Review & Feedback	Submit Review, View Reviews, Edit Review, Delete Review	reviews.txt, ratings.txt	Encapsulation, Inheritance (Verified/Public), Polymorphism (getDisplayCard)

Each component independently satisfies the Acceptable Individual Contribution criteria as specified in the SE1020 project guidelines: minimum 3 CRUD operations, minimum 3 UI pages, proper OOP implementation, file handling, and individual GitHub commit history.

Component 01: User Account Management

Description

This component manages all user-related functionalities for the platform. It handles user registration, authentication, profile management, and account deletion. Two user roles are supported — Regular User and Admin User — modelled using inheritance from a common User base class. All data is persisted in text files. The component enforces session-based role access control to separate customer and admin capabilities throughout the application.

CRUD Operations

- **Create:** Register a new user (Regular or Admin) and persist their details including unique user ID, username, hashed password, email, phone number, and membership type to users.txt.
- **Read:** Search and retrieve user profiles by username, email, or user ID; display user profile page and admin user management list with pagination.
- **Update:** Modify existing user details such as email, phone number, password, or membership tier (Regular/Premium); validate inputs before rewriting the record to users.txt.
- **Delete:** Soft-delete or permanently remove a user account from users.txt; revoke active session and cascade-flag related bookings.

UI Components

- User Registration Page — HTML form with client-side and server-side validation (unique username/email check)
- User Login Page — Authentication form with session management and role-based redirect
- User Profile Page — View and edit personal details (email, phone, password, membership type)
- Admin User Management Panel — Paginated list of all users with search, filter, edit, and delete actions
- Change Password Page — Dedicated secure page for password update with current-password confirmation

OOP Concepts Applied

- **Encapsulation:** User class holds private fields (userId, username, email, passwordHash, phone, membershipType) with public getters/setters; password stored as hash, never plain text.
- **Inheritance:** AdminUser and RegularUser both extend the abstract base class User, inheriting common attributes and overriding role-specific methods such as getPermissions() and getDashboardURL().

- **Polymorphism:** The authenticate() method is overridden in AdminUser (checks admin privilege flag) and RegularUser (standard credential check).
- **Abstraction:** Abstract User class defines the structure and abstract method contracts; concrete subclasses provide specific implementations.
- **Information Hiding:** UserService encapsulates all file I/O logic (read/write/update/delete from users.txt); Servlets call only service methods.

File Handling Operations

- Append new user records to users.txt in pipe-delimited format on registration.
- Read and parse users.txt to validate credentials during login and retrieve profile data.
- Update specific user record by reading all lines, modifying the target, and rewriting users.txt.
- Delete user record by filtering matching line and rewriting file; maintain admins.txt separately.

Additional Features & Scope

- Session management via HttpSession — stores logged-in userId and role; invalidated on logout.
- Duplicate prevention — check existing usernames and emails before writing new registrations.
- Role-based access control — Admin-only pages check session role and redirect unauthorised users.
- Membership tiers (Regular / Premium) affecting booking priority and discount eligibility in other components.
- Input validation on both client (JavaScript) and server (Servlet) sides for all form submissions.

Component 02: Movie & Showtime Management

Description

This component manages the complete movie catalogue and showtime scheduling for the cinema platform. Admin users can add movies, schedule screenings across cinema halls, update existing entries, and remove outdated records. Regular users and guests can browse, search, and filter movies, and view all available showtimes for a chosen film. Inheritance differentiates Featured and Regular movies, each with distinct display priorities and rendering behaviours on the listing page.

CRUD Operations

- **Create:** Add a new movie to movies.txt with full details (title, genre, duration, language, rating, synopsis, cast, poster URL) and schedule one or more showtimes to showtimes.txt with cinema hall, date, time, and total seat count.
- **Read:** Browse all currently screening movies with search and filter capabilities (title, genre, language, rating range); view full movie detail page including synopsis, cast, rating, and all showtimes with seat availability.
- **Update:** Edit movie metadata (synopsis, content rating, poster URL, status) or modify showtime details (date, time, hall assignment, available seat count) through the admin management interface.
- **Delete:** Remove a movie from movies.txt and cascade-delete its associated showtimes from showtimes.txt; flag related bookings in bookings.txt as CANCELLED.

UI Components

- Movie Listing / Home Page — Card-grid display of all Now Showing movies with poster, title, genre, rating, and duration
- Movie Detail Page — Full information page with synopsis, cast, content rating, and a table of available showtimes
- Add / Edit Movie Form (Admin) — Admin form for creating and modifying movie records with full field set
- Showtime Scheduling Page (Admin) — Assign and manage dates, times, halls, and seat counts for a selected movie
- Movie Search & Filter Page — Search by title, genre, language, or release date; filter by status (Now Showing / Coming Soon)

OOP Concepts Applied

- **Encapsulation:** Movie class encapsulates all movie properties as private fields; ShowTime class encapsulates hallId, movieId, date, time, and availableSeats.
- **Inheritance:** FeaturedMovie and RegularMovie extend Movie; FeaturedMovie overrides getDisplayPriority() and getBannerStyle() to achieve promoted display treatment.

- **Polymorphism:** The `getDisplayCard()` method is overridden in `FeaturedMovie` and `RegularMovie` to render different HTML card layouts with and without featured badges.
- **Abstraction:** Abstract Movie class defines core attributes and abstract methods; `MovieService` provides the concrete file-based implementation.
- **Information Hiding:** `MovieService` and `ShowtimeService` encapsulate all .txt file parsing and I/O; Servlets interact only through clean service method APIs.

File Handling Operations

- Append new movie records to `movies.txt` with auto-generated unique movie ID on admin creation.
- Read and parse `movies.txt` on page load to populate the movie listing; filter in-memory by search criteria.
- Update movie records by read-all, modify-target-line, rewrite approach on `movies.txt`.
- Delete movie record from `movies.txt` and all matching showtime records from `showtimes.txt`.
- Read `showtimes.txt` and join with `movies.txt` by `movieId` to display showtime list on the detail page.

Additional Features & Scope

- Movie status flags (Now Showing / Coming Soon / Ended) controlling visibility on the public listing page.
- Multiple showtimes per movie per day supported — one-to-many relationship between `movies.txt` and `showtimes.txt` via `movieId`.
- Cinema hall capacity tracking — seat count in `showtimes.txt` decremented as bookings are created by Component 03.
- Admin-only access guard on all create/edit/delete operations via session role checks.
- Coming Soon page showing upcoming releases with release date countdown.

Component 03: Seat Selection & Booking Management

Description

This component manages the complete ticket booking workflow from seat selection through to booking confirmation and cancellation. Users choose a showtime, select available seats on a visual seat map, review their order summary, and confirm their booking (pending payment by Component 04). Polymorphism is applied to a seat class hierarchy — VIPSeat, PremiumSeat, and StandardSeat — each overriding pricing and display methods. Booking records track status lifecycle: PENDING, CONFIRMED, CANCELLED.

CRUD Operations

- **Create:** Generate a new booking record in bookings.txt with a unique booking reference, userId, showtimeId, selected seatId(s), seat type, total price, booking date, and initial status of PENDING.
- **Read:** Retrieve a user's full booking history filtered by userId from bookings.txt; display real-time seat availability for a given showtime; view full booking details by reference number.
- **Update:** Transition booking status (PENDING to CONFIRMED after successful payment by Component 04; CONFIRMED to CANCELLED on cancellation) and update available seat count in showtimes.txt.
- **Delete/Cancel:** Allow users to cancel eligible bookings (within cancellation window) — update status to CANCELLED, release seats back to available, and trigger refund in Component 04.

UI Components

- Seat Selection Page — Interactive visual seat map with colour-coded status (Available / Selected / Booked / VIP / Premium)
- Booking Summary Page — Order review showing selected seats, showtime info, seat type breakdown, price calculation, and promo code entry field
- Booking Confirmation Page — Success screen with unique booking reference code and printable e-ticket details
- My Bookings Page — Chronological list of all user bookings with status badges (Pending / Confirmed / Cancelled)
- Booking Detail Page — Full detail view of a single booking with seat info, showtime, payment status, and cancel button (if eligible)

OOP Concepts Applied

- **Encapsulation:** Booking class holds private fields (bookingRef, userId, showtimeId, seatIds[], totalPrice, status, bookingDate); Seat class holds seatId, seatType, rowLabel, seatNumber, price, isAvailable.

- **Inheritance:** PremiumBooking and StandardBooking extend Booking — PremiumBooking overrides calculatePrice() to apply premium seat surcharges and weekend pricing.
- **Polymorphism:** VIPSeat, PremiumSeat, and StandardSeat all extend Seat and override getPrice() and getDisplayStyle() methods to return type-specific prices and CSS class names.
- **Abstraction:** IBookingService interface defines createBooking(), cancelBooking(), getBookingHistory() contracts; FileBookingService implements file-based persistence.
- **Information Hiding:** BookingService encapsulates all seat availability checks, booking reference generation, and file I/O; Servlets call only service API methods.

File Handling Operations

- Write new booking records to bookings.txt in structured pipe-delimited format on booking creation.
- Read bookings.txt filtering by userId to load user booking history on My Bookings page.
- Read seats.txt to determine seat type, price, and availability for seat map rendering.
- Update bookings.txt (status field) and showtimes.txt (availableSeats count) on status transitions.
- Implement sequential read-lock-update-write to prevent double-booking of the same seat.

Additional Features & Scope

- Visual seat map with real-time availability — seats marked as booked update on page load from showtimes.txt.
- Multi-seat selection — users can select up to 8 seats in a single booking transaction.
- Cancellation policy enforcement — cancel only permitted more than 2 hours before showtime start.
- Unique booking reference generation in format BK-YYYYMMDD-XXXXXX for each confirmed booking.
- Price breakdown showing base price per seat, seat type surcharge, subtotal, promo discount (from Component 05), and final total.
- Integration hooks: sends total to Component 04 (Payment) on checkout; receives promo discount from Component 05.

Component 04: Payment & Transaction Management

Description

This component handles all payment-related operations on the platform — processing ticket payments, recording transaction histories, managing refunds for cancelled bookings, and providing admins with a full transaction audit view. The design applies the Abstraction and Polymorphism principles through an `IPaymentProcessor` interface, enabling polymorphic support for three simulated payment methods: Credit Card, Debit Card, and Digital Wallet. All transaction data is persisted in text files.

CRUD Operations

- **Create:** Process a payment for a confirmed booking and write a new transaction record to `payments.txt` with `transactionId`, `bookingRef`, `userId`, `amount`, `paymentMethod`, `status` (SUCCESS/FAILED), and `timestamp`.
- **Read:** Retrieve full transaction history for a user by `userId`; view a specific transaction by `transactionId`; display payment receipt on the confirmation page and booking detail page.
- **Update:** Process refunds for cancelled bookings by updating the transaction status to `REFUNDED` in `payments.txt` and logging the `refundTimestamp` and `refundAmount`.
- **Delete:** Allow admins to permanently remove erroneous or test transaction records from `payments.txt` after confirmation; log deletion to an audit file.

UI Components

- Payment Checkout Page — Order summary with total amount and payment method selector (Credit Card / Debit Card / Digital Wallet)
- Payment Confirmation / Receipt Page — Success view with `transactionId`, amount paid, booking reference, and print receipt button
- Transaction History Page — Paginated list of all user payments with date, amount, method, and status badge (Success / Refunded / Failed)
- Refund Request / Status Page — User-facing view showing refund eligibility and refund status for a cancelled booking
- Admin Transaction Dashboard — Full platform-wide transaction audit view with search by `bookingRef`/`userId`, date range filter, and total revenue summary

OOP Concepts Applied

- **Encapsulation:** Transaction class encapsulates `transactionId`, `bookingRef`, `userId`, `amount`, `paymentMethod`, `status`, `createdAt`, and `refundedAt` as private fields.
- **Inheritance:** `CreditCardPayment`, `DebitCardPayment`, and `WalletPayment` all extend abstract `Payment` class, inheriting common `processPayment()` flow and overriding method-specific validation.

- **Polymorphism:** processPayment() is overridden in each subclass to simulate different validation and processing logic per payment method type.
- **Abstraction:** IPaymentProcessor interface defines processPayment(), processRefund(), and generateReceipt() contracts — Servlets interact only through this interface, decoupled from concrete implementations.
- **Information Hiding:** PaymentService hides all file I/O, transaction ID generation, and status management; only clean method signatures exposed externally.

File Handling Operations

- Append new transaction records to payments.txt in pipe-delimited format after payment processing.
- Read payments.txt filtering by userId to load transaction history page.
- Update specific transaction record in payments.txt to REFUNDED status with refund timestamp.
- Maintain transactions.txt as an append-only audit log of all payment events (including failed attempts).
- Admin reads all records from payments.txt for dashboard, applying date-range and status filters in-memory.

Additional Features & Scope

- Simulated payment processing — configurable success/failure toggle for demo and viva demonstration purposes.
- Auto-refund triggering — when Component 03 cancels a booking, Component 04 is invoked to process the refund automatically.
- Transaction reference format: TXN-YYYYMMDD-XXXXX for all created transactions.
- Payment receipt display with formatted printable layout after successful payment.
- Revenue analytics on admin dashboard: total daily/monthly revenue, total refunds issued, and payment method breakdown.
- Integration hook — updates booking status to CONFIRMED in bookings.txt on successful payment completion.

Component 05: Promotion & Discount Management

Description

This component manages the full lifecycle of promotional codes and discount campaigns on the platform. Admins can create time-limited promotions with various discount types, set usage limits, and manage their active status. Users apply promo codes at checkout (on the Booking Summary page of Component 03) to receive a price reduction. The module applies inheritance to model distinct promotion types (percentage-based, fixed-amount, seasonal), with polymorphism used to calculate the discount amount differently per type.

CRUD Operations

- **Create:** Admin creates a new promotion with a unique promo code, discount type (Percentage/Fixed Amount), discount value, validity start/end dates, maximum usage count, minimum order amount condition, and active status — persisted to promotions.txt.
- **Read:** Retrieve and validate a promo code entered by a user (check code exists, is active, not expired, not exceeded usage limit, and order meets minimum amount); list all promotions on the admin management panel with full details.
- **Update:** Modify promotion details (extend dates, change value, update usage limit, toggle active/inactive status) and auto-increment usageCount each time a code is successfully applied at checkout.
- **Delete:** Remove expired or invalid promotions from promotions.txt; prevent deletion of promotions with active pending bookings using them.

UI Components

- Promotions Management Page (Admin) — Full CRUD panel listing all promotions with create, edit, activate/deactivate, and delete actions
- Create / Edit Promotion Form (Admin) — Detailed form with all promotion fields: type, value, dates, usage limit, minimum amount, and applicable movie/category conditions
- Active Promotions / Deals Page (Public) — Customer-facing listing of currently valid and active promotional offers
- Promo Code Validation Widget — Inline promo code entry integrated into the Booking Summary page (Component 03) with real-time validation feedback
- Promotion Usage Analytics Page (Admin) — Dashboard showing total redemptions, total discount value granted, remaining usages, and most popular promo codes

OOP Concepts Applied

- **Encapsulation:** Promotion class holds private fields (promoCode, discountType, discountValue, startDate, endDate, usageLimit, usageCount, minOrderAmount, isActive).

- **Inheritance:** PercentageDiscount and FixedAmountDiscount extend abstract Promotion; SeasonalPromotion extends PercentageDiscount with additional date-range validation overrides.
- **Polymorphism:** The calculateDiscount(double originalPrice) method is overridden in each subclass — PercentageDiscount applies a percentage reduction; FixedAmountDiscount subtracts a flat amount; SeasonalPromotion applies bonus multipliers during peak periods.
- **Abstraction:** Abstract Promotion class defines core fields and the abstract calculateDiscount() method, enforcing a consistent interface for all promotion types.
- **Information Hiding:** PromotionService encapsulates all file I/O, expiry checking, usage-limit enforcement, and discount calculation; calling code receives only the discount result.

File Handling Operations

- Append new promotion records to promotions.txt in structured format on admin creation.
- Read and parse promotions.txt to validate a user's promo code — checking code, status, dates, and usage count.
- Update the usageCount field in promotions.txt each time a promo code is successfully applied at checkout.
- Update promotion records for admin edits (date changes, value changes, status toggles) using read-all, modify, rewrite approach.
- Filter and read promotions.txt to display only active and unexpired promotions on the public deals page.

Additional Features & Scope

- Promo code uniqueness check on creation — rejects duplicate codes before writing to promotions.txt.
- Auto-invalidation — promotions past their endDate or with usageCount reaching usageLimit are treated as inactive on validation.
- Minimum order amount enforcement — discount only applied if the booking total meets the specified threshold.
- Category/movie-specific promotions — optional condition to restrict promo to a specific genre or movie title.
- Integration with Component 03 Booking Summary and Component 04 Payment for seamless discount application.
- Promotion performance report: total codes created, total redemptions, total platform discount value issued.

Component 06: Review, Rating & Feedback Management

Description

This component enables users to submit star ratings and written reviews for movies they have watched (CONFIRMED and past bookings), manage their own review content, and enables admins to moderate submissions. The module supports verified purchaser reviews (linked to confirmed bookings) and general reviews, modelled through inheritance. Polymorphism is used to differentiate display and moderation behaviour. Aggregated movie ratings are computed and stored in ratings.txt to power the star rating display on movie pages.

CRUD Operations

- **Create:** Authenticated users with a confirmed past booking can submit a new review for that movie — including a star rating (1 to 5), review title, review body text, and optional spoiler flag — persisted to reviews.txt with auto-generated reviewId.
- **Read:** Display all approved reviews for a movie on the Movie Detail page (Component 02 integration); show average star rating and rating distribution bars; list a user's own reviews on their profile page; load all reviews with status filters on the admin moderation panel.
- **Update:** Allow the review author to edit their own review (rating, title, body) within 7 days of submission; allow admins to change review status (Pending / Approved / Rejected / Hidden) from the moderation panel.
- **Delete:** Review authors can delete their own reviews; admins can delete any review including flagged/reported ones; deletion triggers recalculation of the movie's average rating in ratings.txt.

UI Components

- Movie Review Section — Embedded review listing on the Movie Detail Page with average star rating widget, rating distribution bar chart, and individual review cards sorted by date or helpfulness
- Submit Review Page — Review form with star rating selector widget, title input, review body textarea, spoiler toggle, and submission validation (booking verification check)
- My Reviews Page — User profile section showing all submitted reviews with edit and delete options, and review status badges
- Admin Review Moderation Panel — Paginated admin view of all reviews filterable by status (Pending / Approved / Reported / Hidden) with approve, reject, hide, and delete actions
- Movie Ratings Summary Widget — Reusable component showing average rating (to 1 decimal), 5-star-to-1-star distribution bars, and total review count — embedded in the movie listing and detail pages

OOP Concepts Applied

- **Encapsulation:** Review class holds private fields (reviewId, movieId, userId, rating, title, body, submissionDate, editDate, status, isSpoiler, helpfulCount, isVerified).
- **Inheritance:** VerifiedReview (submitted by users with a confirmed booking) extends PublicReview; VerifiedReview overrides getVerifiedBadge() and carries higher weight in average rating calculation.
- **Polymorphism:** The getDisplayCard() method is overridden in VerifiedReview and PublicReview to render different card layouts — VerifiedReview cards display a 'Verified Booking' trust badge.
- **Abstraction:** IReviewService interface defines submitReview(), editReview(), deleteReview(), getMovieReviews() contracts; FileReviewService provides the file-based concrete implementation.
- **Information Hiding:** ReviewService manages all reviews.txt and ratings.txt read/write operations, average rating computation, and status filtering — exposing only clean method APIs to Servlets.

File Handling Operations

- Append new review records to reviews.txt in pipe-delimited format with auto-incremented reviewId on submission.
- Read and parse reviews.txt filtering by movieId to display movie reviews on the detail page.
- Read and filter reviews.txt by userId to populate the My Reviews page.
- Update specific review record in reviews.txt on author edit or admin status change (read-all, modify, rewrite).
- Delete review record by filtering out matching reviewId and rewriting reviews.txt; recalculate and update ratings.txt.
- Read and write ratings.txt to maintain per-movie average star rating — updated on every review submission, edit, or deletion.

Additional Features & Scope

- Verified booking check — review submission only permitted for users with a CONFIRMED and past-dated booking for the film.
- Spoiler flag — reviews marked as spoilers display only a blurred placeholder with a 'Reveal Spoiler' toggle on the listing.
- Helpfulness voting — users can upvote reviews as Helpful; helpfulCount stored in reviews.txt and used for relevance sorting.
- Anti-duplicate enforcement — system prevents the same user from submitting more than one review per movie.
- User reporting — readers can flag inappropriate reviews, triggering status change to REPORTED for admin review.
- Average rating recalculation — ratings.txt updated atomically whenever a review is created, edited, or deleted; power the star display across the platform.

10. Non-Functional Requirements

Category	Requirement
Performance	All page loads must respond within 2 seconds under normal file I/O conditions.
Usability	UI must be responsive on desktop and tablet screen sizes using Bootstrap 5 or Tailwind CSS.
Security	Passwords must not be stored in plain text; session-based role checks must prevent unauthorised access to admin pages.
Maintainability	Code must follow standard Java naming conventions with Javadoc comments on all public classes and methods.
Reliability	All file write operations must handle IOExceptions gracefully and not corrupt data files on error.
Portability	Application must run on any OS with JDK 17+ and Apache Tomcat installed.
Version Control	Each member must have a minimum of 5 meaningful, descriptive GitHub commits demonstrating progressive development.
Documentation	Each component requires a class diagram; the final report must include full GitHub commit history screenshots.

11. Marking Criteria Mapping

Assessment Criterion	Max Marks	How Addressed in This SRS
CRUD Operations	30	All 6 components implement all 4 CRUD operations with file-based or DB persistence.
OOP Concepts	20	All 5 OOP principles applied in every component with concrete class hierarchies and interfaces.
File Handling	10	Dedicated file read/write operations for each component with structured .txt data files.
User Interface Design	10	Minimum 5 responsive UI pages per component using Bootstrap or Tailwind CSS.
Individual Contribution & GitHub	10	Each component owned by one member; individual commits tracked on GitHub.
Presentation & Viva	10	Each member demonstrates their component; Java backend and OOP must be explainable.

Assessment Criterion	Max Marks	How Addressed in This SRS
Documentation	10	Class diagram and final report with GitHub commit history required per component.
TOTAL	100	Full coverage of all assessment criteria across all 6 components.

12. Deliverables & GitHub Guidelines

12.1 Repository Folder Structure

- **src/main/java/com/cinebooking/component01/**: UserAccountManagement — servlets, services, models
- **src/main/java/com/cinebooking/component02/**: MovieShowtimeManagement
- **src/main/java/com/cinebooking/component03/**: SeatBookingManagement
- **src/main/java/com/cinebooking/component04/**: PaymentTransactionManagement
- **src/main/java/com/cinebooking/component05/**: PromotionDiscountManagement
- **src/main/java/com/cinebooking/component06/**: ReviewRatingManagement
- **src/main/webapp/WEB-INF/views/**: JSP files for all components
- **data/**: users.txt, movies.txt, showtimes.txt, bookings.txt, payments.txt, promotions.txt, reviews.txt, ratings.txt
- **docs/**: Class diagrams (PDF/PNG per component), Final Report (PDF)

12.2 Deliverables Checklist

#	Deliverable	Format	Responsible
1	Source code — all .java, .jsp, and config files	GitHub Repository	All members
2	Class diagram for assigned component	PDF or PNG in /docs folder	Each member individually
3	Final report with GitHub commit history screenshots	PDF or DOCX	Group collectively
4	Sample data files (all .txt files pre-populated)	In /data folder on GitHub	All members
5	Viva — live demo and explanation of individual component	Live presentation	Each member individually

12.3 Commit Message Best Practices

- [C01] Add user registration servlet and file write logic
- [C02] Implement movie search filter by genre and status
- [C03] Add seat availability check before booking creation
- Use imperative present tense: 'Add', 'Fix', 'Implement', 'Update' — not 'Added' or 'Adding'.
- Commit progressively — never submit all work in a single commit; the history must show real development.
- Minimum 5 commits per member; recommended 10+ for full marks on individual contribution.

End of Software Requirements Specification

SE1020 – Object Oriented Programming | Online Movie Ticket Booking Platform | Version 1.0